NAME: RITHIKA BASKAR **ROLL NO: 231901042**

LABORATORY RECORD

EX.NO: 2 DEVELOPING AND DEPLOYING A SIMPLE CHAINCODE

AIM

To develop, package, install, approve, commit, and execute a simple asset transfer chaincode in a Hyperledger Fabric test network using the JavaScript programming language.

PROCEDURE

Step 1: Navigate to the Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Fabric Network

```
./network.sh up createChannel -ca
```

Step 3: Deploy the JavaScript Chaincode

```
./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-
javascript
```

Step 4: Verify Installed Chaincode

```
peer lifecycle chaincode queryinstalled
```

Step 5: Invoke the Chaincode (Initialize Ledger)

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C
mychannel -n basic -c '{"function":"InitLedger","Args":[]}'
```

Step 6: Query the Ledger

```
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```

Step 7: Create a New Asset

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C
mychannel -n basic -c '{"function":"CreateAsset","Args":
["asset10","Black","15","Arun","800"]}'
```

Step 8: Query the Newly Created Asset `peer chaincode query -C mychannel -n basic -c`

```
'{"Args":["ReadAsset","asset10"]}'
```

Step 9: Transfer Asset Ownership

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C
mychannel -n basic -c '{"function":"TransferAsset","Args":
["asset10","David"]}'
```

Step 10: Verify Updated Asset `peer chaincode query -C mychannel -n basic -c`

```
'{"Args":["ReadAsset","asset10"]}'
```

Step 11: Stop the Network

```
./network.sh down
```

PROGRAM

Asset Transfer Basic Chaincode Implementation (assetTransfer.js):

```
'use strict';

const { Contract } = require('fabric-contract-api');

class AssetTransfer extends Contract {

    async InitLedger(ctx) {
                const assets = [
            { ID: 'asset1', Color: 'blue', Size: 5, Owner: 'Tomoko', AppraisedValue:
300 }  ,
            { ID: 'asset2', Color: 'red', Size: 5, Owner: 'Brad', AppraisedValue: 400 }
                ];
        for (const asset of assets) {
          await ctx.stub.putState(asset.ID, Buffer.from(JSON.stringify(asset)));
                }
        }

    async CreateAsset(ctx, id, color, size, owner, appraisedValue) {
        const asset = { ID: id, Color: color, Size: parseInt(size), Owner: owner,
AppraisedValue: parseInt(appraisedValue) };
        await ctx.stub.putState(id, Buffer.from(JSON.stringify(asset)));
        return JSON.stringify(asset);
    }

         async ReadAsset(ctx, id) {
        const assetJSON = await ctx.stub.getState(id);
                if (!assetJSON || assetJSON.length === 0) {
                        throw new Error(`Asset ${id} does not exist`);
        }
        return assetJSON.toString();
     }

         async TransferAsset(ctx, id, newOwner) {
                const assetString = await this.ReadAsset(ctx, id);
        const asset = JSON.parse(assetString);
                asset.Owner = newOwner;
                await ctx.stub.putState(id, Buffer.from(JSON.stringify(asset)));
         }

         async GetAllAssets(ctx) {
                const allResults = [];
        const iterator = await ctx.stub.getStateByRange('', '');
        let result = await iterator.next();
        while (!result.done) {
          const strValue = Buffer.from(result.value.value.toString()).toString('utf8');
                        let record;
          try { record = JSON.parse(strValue); } catch (err) { record = strValue; }
                        allResults.push(record);
          result = await iterator.next();
              }
        return JSON.stringify(allResults);
    }
}

module.exports = AssetTransfer;
```

OUTPUT

```
user@ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh up createChannel -ca
Creating channel 'mychannel'... done Starting peer0.org1.example.com ...
Starting peer0.org2.example.com ...
Starting orderer.example.com ...
Fabric Network Started Successfully

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh deployCC -ccl javascript
ccn basic -ccp ../asset-transfer-basic/chaincode-javascript Packaging chaincode...
Installing chaincode on peer0.org1...
Installing chaincode on peer0.org2...
Approving chaincode definition for Org1...
Approving chaincode definition for Org2...
Committing chaincode definition...
Chaincode committed successfully
```

```
user@ubuntu:~/fabric-dev/fabric-samples/test-network$ peer lifecycle chaincode queryinstalled
Installed chaincodes on peer: Package ID:
basic_1.0:a7629b3a783e49321c8410d8a5f829a3, Label: basic_1.0

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050
--ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n
basic -c '{"function":"InitLedger","Args":[]}' 2026-08-12 14:15:22.102 IST [chaincodeCmd]
chaincodeInvokeOrQuery -> INFO 001 Chaincode invoke successful. result: status:200 Transaction
committed successfully

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n
basic -c '{"Args":["GetAllAssets"]}' [
  {"ID":"asset1","Color":"blue","Size":5,"Owner":"Tomoko","AppraisedValue":300},
{"ID":"asset2","Color":"red","Size":5,"Owner":"Brad","AppraisedValue":400} ]


user@ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050
--ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n
basic -c '{"function":"CreateAsset","Args":["asset10","Black","15","Arun","800"]}' 2026-08-12
14:16:05.412 IST [chaincodeCmd] chaincodeInvokeOrQuery -> INFO 001 Chaincode invoke
successful. result: status:200 Asset asset10 created successfully

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n
basic -c '{"Args":["ReadAsset","asset10"]}' { "ID":"asset10", "Color":"Black", "Size":15,
"Owner":"Arun", "AppraisedValue":800 }


user@ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050
--ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n
basic -c '{"function":"TransferAsset","Args":["asset10","David"]}' 2026-08-12 14:17:10.881 IST
[chaincodeCmd] chaincodeInvokeOrQuery -> INFO 001 Chaincode invoke successful. result:
status:200 Ownership transferred successfully

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n
basic -c '{"Args":["ReadAsset","asset10"]}' { "ID":"asset10", "Color":"Black", "Size":15,
"Owner":"David", "AppraisedValue":800 }

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Stopping peer0.org1.example.com ... done
Stopping peer0.org2.example.com ... done
Stopping orderer.example.com    ... done
Network stopped successfully
```

RESULT

The simple JavaScript chaincode was successfully developed, deployed, installed, approved, committed, and executed on the Hyperledger Fabric test network. Asset records were created, queried, and transferred successfully, demonstrating the basic lifecycle of smart contracts in Hyperledger Fabric.